

Fonic HiFi Technical Specification

File System

The Fonic HiFi application follows a modular architecture with clear separation of concerns. The file system structure is designed to support maintainability, testability, and scalability.

Front-End Repository Structure

```
FonicHiFi/
├── App/
│   ├── FonicHiFiApp.swift      # Main app entry point
│   ├── AppDelegate.swift      # App lifecycle management
│   ├── SceneDelegate.swift    # Scene management for iOS
│   └── AppConfiguration.swift  # Global app configuration
├── Presentation/              # Presentation Layer
│   ├── Views/                 # SwiftUI Views
│   │   ├── Library/          # Library browsing views
│   │   │   ├── LibraryView.swift  # Main library view
│   │   │   ├── ArtistListView.swift # Artists browsing view
│   │   │   ├── AlbumListView.swift # Albums browsing view
│   │   │   ├── GenreListView.swift # Genres browsing view
│   │   │   ├── TrackListView.swift # Tracks browsing view
│   │   │   ├── QualityFilterView.swift # Audio quality filter view
│   │   │   └── Components/      # Reusable library components
│   │   ├── Player/           # Playback interface views
│   │   │   ├── PlayerView.swift  # Main player view
│   │   │   ├── MiniPlayerView.swift # Mini player for navigation
│   │   │   ├── WaveformView.swift # Audio waveform visualization
│   │   │   ├── PlaybackControlsView.swift # Playback controls
│   │   │   ├── QueueView.swift   # Playback queue management
│   │   │   └── Components/      # Reusable player components
│   │   ├── Playlists/        # Playlist management views
│   │   │   ├── PlaylistsView.swift # Playlists overview
│   │   │   ├── PlaylistDetailView.swift # Individual playlist view
│   │   │   ├── SmartPlaylistEditorView.swift # Smart playlist editor
│   │   │   └── Components/      # Reusable playlist components
│   │   └── Metadata/         # Metadata editing views
│   │       └── MetadataEditorView.swift # Metadata editor main view
```

```

graph TD
    Root[Domain/Models/] --- BatchEditorView[BatchEditorView.swift # Batch editing interface]
    Root --- ArtworkEditorView[ArtworkEditorView.swift # Artwork management]
    Root --- Components1[Components/ # Reusable metadata components]
    Root --- Settings1[Settings/ # App settings views]
    Settings1 --- SettingsView[SettingsView.swift # Main settings view]
    Settings1 --- AudioSettingsView[AudioSettingsView.swift # Audio configuration]
    Settings1 --- LibrarySettingsView[LibrarySettingsView.swift # Library management]
    Settings1 --- AppearanceSettingsView[AppearanceSettingsView.swift # UI customization]
    Settings1 --- Components2[Components/ # Reusable settings components]
    Root --- Common[Common/ # Shared UI components]
    Common --- AudioQualityBadge[AudioQualityBadge.swift # Audio quality indicator]
    Common --- LoadingView[LoadingView.swift # Loading state view]
    Common --- ErrorView[ErrorView.swift # Error state view]
    Common --- Ellipsis1[...] # Other shared components
    Root --- ViewModels[ViewModels/ # MVVM ViewModels]
    ViewModels --- Library[Library/ # Library ViewModels]
    Library --- LibraryViewModel[LibraryViewModel.swift # Main library ViewModel]
    Library --- ArtistListViewModel[ArtistListViewModel.swift # Artists ViewModel]
    Library --- AlbumListViewModel[AlbumListViewModel.swift # Albums ViewModel]
    Library --- Ellipsis2[...] # Other library ViewModels
    ViewModels --- Player[Player/ # Player ViewModels]
    Player --- PlayerViewModel[PlayerViewModel.swift # Main player ViewModel]
    Player --- WaveformViewModel[WaveformViewModel.swift # Waveform visualization]
    ViewModels --- Ellipsis3[...] # Other player ViewModels
    ViewModels --- Playlists[Playlists/ # Playlist ViewModels]
    Playlists --- PlaylistsViewModel[PlaylistsViewModel.swift # Playlists ViewModel]
    Playlists --- SmartPlaylistViewModel[SmartPlaylistViewModel.swift # Smart playlist ViewModel]
    Playlists --- Ellipsis4[...] # Other playlist ViewModels
    ViewModels --- Metadata[Metadata/ # Metadata ViewModels]
    Metadata --- MetadataEditorViewModel[MetadataEditorViewModel.swift # Metadata editor ViewModel]
    Metadata --- Ellipsis5[...] # Other metadata ViewModels
    ViewModels --- Settings2[Settings/ # Settings ViewModels]
    Settings2 --- SettingsViewModel[SettingsViewModel.swift # Settings ViewModel]
    Settings2 --- Ellipsis6[...] # Other settings ViewModels
    ViewModels --- UIState[UIState/ # UI State Management]
    UIState --- AppState[AppState.swift # Global app state]
    UIState --- NavigationState[NavigationState.swift # Navigation state]
    UIState --- ThemeManager[ThemeManager.swift # UI theme management]
    Root --- Domain[Domain/ # Domain Layer]
    Domain --- Models[Models/ # Domain Models]
  
```

The diagram illustrates the project structure of a Swift application, organized into a hierarchical tree. The root folder is **Domain/Models/**, which serves as the base for the Domain Layer. The structure is as follows:

- Domain/Models/** (Domain Layer)
 - BatchEditorView.swift** # Batch editing **interface**
 - ArtworkEditorView.swift** # Artwork management
 - Components/** # Reusable metadata components
 - Settings/** # App settings views
 - SettingsView.swift** # Main settings view
 - AudioSettingsView.swift** # Audio configuration
 - LibrarySettingsView.swift** # Library management
 - AppearanceSettingsView.swift** # UI customization
 - Components/** # Reusable settings components
 - Common/** # Shared UI components
 - AudioQualityBadge.swift** # Audio quality indicator
 - LoadingView.swift** # Loading state view
 - ErrorView.swift** # Error state view
 - ...** # Other shared components
 - ViewModels/** # MVVM ViewModels
 - Library/** # Library ViewModels
 - LibraryViewModel.swift** # Main **library** ViewModel
 - ArtistListViewModel.swift** # Artists ViewModel
 - AlbumListViewModel.swift** # Albums ViewModel
 - ...** # Other **library** ViewModels
 - Player/** # Player ViewModels
 - PlayerViewModel.swift** # Main player ViewModel
 - WaveformViewModel.swift** # Waveform visualization
 - ...** # Other player ViewModels
 - Playlists/** # Playlist ViewModels
 - PlaylistsViewModel.swift** # Playlists ViewModel
 - SmartPlaylistViewModel.swift** # Smart playlist ViewModel
 - ...** # Other playlist ViewModels
 - Metadata/** # Metadata ViewModels
 - MetadataEditorViewModel.swift** # Metadata editor ViewModel
 - ...** # Other metadata ViewModels
 - Settings/** # Settings ViewModels
 - SettingsViewModel.swift** # Settings ViewModel
 - ...** # Other settings ViewModels
 - UIState/** # UI State Management
 - AppState.swift** # Global app state
 - NavigationState.swift** # Navigation state
 - ThemeManager.swift** # UI theme management
 - Domain/** # Domain Layer
 - Models/** # Domain Models

```

├── Track.swift           # Audio track model
├── Album.swift          # Album model
├── Artist.swift         # Artist model
├── Playlist.swift       # Playlist model
├── SmartPlaylist.swift  # Smart playlist model
├── AudioFormat.swift    # Audio format model
├── ...                  # Other domain models
├── UseCases/           # Business Logic Use Cases
│   ├── Library/        # Library use cases
│   │   ├── ImportMusicUseCase.swift # Music import logic
│   │   ├── ScanLibraryUseCase.swift # Library scanning logic
│   │   └── ...          # Other library use cases
│   ├── Playback/       # Playback use cases
│   │   ├── PlayTrackUseCase.swift   # Track playback logic
│   │   ├── QueueManagementUseCase.swift # Queue management logic
│   │   └── ...                      # Other playback use cases
│   ├── Playlists/      # Playlist use cases
│   │   ├── CreatePlaylistUseCase.swift # Playlist creation logic
│   │   ├── SmartFilterUseCase.swift   # Smart filtering logic
│   │   └── ...                      # Other playlist use cases
│   └── Metadata/       # Metadata use cases
│       ├── EditMetadataUseCase.swift # Metadata editing logic
│       ├── BatchEditUseCase.swift   # Batch editing logic
│       └── ...                      # Other metadata use cases
├── Interfaces/         # Repository Interfaces
│   ├── ILibraryRepository.swift      # Library repository interface
│   ├── IPlaybackRepository.swift     # Playback repository interface
│   ├── IPlaylistRepository.swift     # Playlist repository interface
│   └── IMetadataRepository.swift     # Metadata repository interface
├── Data/               # Data Layer
│   ├── Repositories/   # Repository Implementations
│   │   ├── LibraryRepository.swift    # Library repository implementation
│   │   ├── PlaybackRepository.swift   # Playback repository implementation
│   │   ├── PlaylistRepository.swift   # Playlist repository implementation
│   │   └── MetadataRepository.swift   # Metadata repository implementation
│   ├── DataSources/    # Data Sources
│   │   ├── Local/      # Local data sources
│   │   │   ├── SwiftDataSource.swift  # SwiftData source
│   │   │   ├── FileSystemDataSource.swift # File system source
│   │   │   └── UserDefaultsDataSource.swift # User defaults source
│   │   └── Remote/     # Remote data sources (future)
│   │       ├── CloudDataSource.swift   # Cloud storage source

```

- └── StreamingDataSource.swift # Streaming service source
- └── DTOs/ # Data Transfer Objects
 - └── TrackDTO.swift # Track data transfer object
 - └── AlbumDTO.swift # Album data transfer object
 - └── ... # Other DTOs
- └── Mappers/ # Data Mappers
 - └── TrackMapper.swift # Track entity-model mapper
 - └── AlbumMapper.swift # Album entity-model mapper
 - └── ... # Other mappers
- └── Core/ # Core Services
 - └── Audio/ # Audio Engine Services
 - └── AudioEngine/ # Audio playback engine
 - └── AudioEngineService.swift # Main audio engine service
 - └── AVAudioEngineAdapter.swift # AVAudioEngine adapter
 - └── SFBAudioEngineAdapter.swift # SFBAudioEngine adapter
 - └── FFmpegAdapter.swift # FFmpeg adapter
 - └── AudioProcessing/ # Audio processing services
 - └── EqualizerService.swift # Equalizer implementation
 - └── GainControlService.swift # Gain control implementation
 - └── WaveformGeneratorService.swift # Waveform generation
 - └── AudioSession/ # Audio session management
 - └── AudioSessionService.swift # iOS audio session service
 - └── HardwareDetectionService.swift # DAC detection service
- └── Metadata/ # Metadata Services
 - └── TagParserService.swift # Audio tag parsing service
 - └── TagWriterService.swift # Audio tag writing service
 - └── CueSheetService.swift # CUE sheet parsing service
- └── FileSystem/ # File System Services
 - └── FileManagerService.swift # File management service
 - └── ImportService.swift # File **import** service
 - └── StorageMonitorService.swift # Storage monitoring service
- └── Background/ # Background Processing
 - └── BackgroundTaskService.swift # Background task service
 - └── IndexingService.swift # Background indexing service
- └── Utils/ # Utilities
 - └── Extensions/ # Swift extensions
 - └── SwiftUI+Extensions.swift # SwiftUI extensions
 - └── Foundation+Extensions.swift # Foundation extensions
 - └── ... # Other extensions
- └── Helpers/ # Helper utilities



```

FonicHiFiAudioKit/
├── Sources/
│   ├── AudioFormats/                # Audio Format Support
│   │   ├── FLACDecoder.swift        # FLAC decoding implementation
│   │   ├── ALACDecoder.swift        # ALAC decoding implementation
│   │   ├── DSDDecoder.swift         # DSD decoding implementation
│   │   └── ...                       # Other format decoders
│   ├── DSP/                         # Digital Signal Processing
│   │   ├── Equalizer/               # Equalizer implementation
│   │   │   ├── EqualizerNode.swift  # Equalizer processing node
│   │   │   └── EqualizerPresets.swift # Equalizer presets
│   └── GainControl/                 # Gain control implementation

```

```

├── GainNode.swift           # Gain processing node
├── ReplayGain.swift        # ReplayGain implementation
├── Effects/                # Audio effects (future)
│   ├── CrossfadeProcessor.swift # Crossfade implementation
│   └── ...                 # Other effects
├── Analysis/              # Audio Analysis
│   ├── WaveformGenerator.swift # Waveform generation
│   ├── SpectrumAnalyzer.swift  # Spectrum analysis
│   └── ...                 # Other analysis tools
├── Hardware/              # Hardware Integration
│   ├── DACDetector.swift      # DAC detection implementation
│   ├── OutputConfigurator.swift # Output configuration
│   ├── BitPerfectValidatorService.swift # Service to validate bit-perfect output
│   └── ...                 # Other hardware integrations
├── Dependencies/          # Third-party Dependencies
│   ├── FFmpegKit/           # FFmpeg integration
│   ├── SFBAudioEngine/      # SFBAudioEngine integration
│   └── TagLib/               # TagLib integration
├── Tests/                 # Test Suite
│   ├── AudioFormatsTests/    # Format decoder tests
│   ├── DSPTests/            # DSP tests
│   └── ...                 # Other tests

```

Database Schema (SwiftData)

// Core data models for SwiftData

```

@Model
class TrackEntity {
    var id: UUID
    var title: String
    var artist: String
    var albumTitle: String
    var albumArtist: String?
    var composer: String?
    var genre: String?
    var year: Int?
    var trackNumber: Int?
    var discNumber: Int?
    var duration: Double
    var filePath: String
    var fileSize: Int64
    var dateAdded: Date
    var dateModified: Date

```

```

var playCount: Int
var lastPlayed: Date?
var format: String
var bitDepth: Int?
var sampleRate: Int?
var bitrate: Int?
var isLossless: Bool
var isHiRes: Bool
var albumArtwork: Data?
var waveformData: Data?
var lyrics: String?
var isOffline: Bool
var sourceType: String // "local", "cloud", "streaming"

// Relationships
var album: AlbumEntity?
var artistEntity: ArtistEntity?
var playlists: [PlaylistTrackEntity]?
}

```

```

@Model
class AlbumEntity {
    var id: UUID
    var title: String
    var artist: String
    var year: Int?
    var genre: String?
    var artwork: Data?
    var dateAdded: Date

    // Relationships
    var tracks: [TrackEntity]?
    var artistEntity: ArtistEntity?
}

```

```

@Model
class ArtistEntity {
    var id: UUID
    var name: String
    var artwork: Data?

    // Relationships
    var albums: [AlbumEntity]?
    var tracks: [TrackEntity]?
}

```

```

@Model
class PlaylistEntity {
    var id: UUID
    var name: String
    var dateCreated: Date
    var dateModified: Date
}

```

```

var isSmartPlaylist: Bool
var smartRules: Data? // JSON serialized rules for smart playlists

// Relationships
var tracks: [PlaylistTrackEntity]?
}

@Model
class PlaylistTrackEntity {
    var id: UUID
    var position: Int
    var dateAdded: Date

    // Relationships
    var track: TrackEntity?
    var playlist: PlaylistEntity?
}

@Model
class EqualizerPresetEntity {
    var id: UUID
    var name: String
    var bands: Data // JSON serialized equalizer bands
    var isDefault: Bool
}

```

Feature Specifications

Feature 1: Core Music Library

Feature Goal

Provide a comprehensive local music library that supports importing, organizing, and playing high-resolution audio files with full metadata management capabilities.

API Relationships

- Interfaces with iOS Files app via FileProvider framework
- Integrates with TagLib for metadata parsing
- Utilizes SFBAudioEngine and FFmpegKit for extended format support
- Leverages SwiftData with Core Data fallback for metadata indexing

Detailed Feature Requirements

1. **Music Import and Organization**
2. Support importing music from Files app, external drives, and local storage

3. Scan, index, and parse metadata from audio files (ID3, Vorbis, .cue)
4. Organize library with sorted views (Artists, Albums, Genres, File Quality)
5. Support for FLAC, ALAC, AIFF, WAV, APE, DSD formats with priority on FLAC and ALAC
6. Parse and edit embedded tags
7. Support libraries up to 100,000+ tracks or 2TB+ storage

8. Library Browsing and Navigation

9. Provide hierarchical browsing by Artist → Album → Track
10. Enable flat browsing by Albums, Tracks, Genres
11. Implement search functionality across all metadata fields
12. Display audio quality indicators and format badges
13. Support sorting by various metadata attributes
14. Implement filtering by audio quality, format, and other attributes

15. Metadata Management

16. Parse and display complete metadata from audio files
17. Support for album artwork display and extraction
18. Handle embedded lyrics and display during playback
19. Support for cue sheets and track splitting
20. Provide batch editing capabilities for multiple files

21. Performance Optimization

22. Implement efficient indexing for large libraries
23. Use background processing for library scanning and waveform generation
24. Implement lazy loading for artwork and detailed metadata
25. Cache frequently accessed data for performance
26. Support incremental updates to library

27. User Experience

28. Provide clear visual indicators for audio quality and format
29. Implement smooth scrolling and transitions in library views
30. Support dark mode UI with minimalist aesthetic
31. Ensure accessibility compliance (VoiceOver, Dynamic Type)
32. Provide clear feedback during import and scanning operations

Detailed Implementation Guide

1. System Architecture

Library Manager Service ```swift protocol LibraryManagerService { // Library scanning and indexing func scanLibrary() async throws -> LibraryScanResult func importFiles(from url: URL) async throws -> ImportResult func updateLibrary() async throws -> LibraryScanResult

```
// Library access
func getArtists(sortBy: ArtistSortOption) async throws -> [Artist]
func getAlbums(sortBy: AlbumSortOption, filterBy: AlbumFilter?) async throws -> [Album]
func getTracks(sortBy: TrackSortOption, filterBy: TrackFilter?) async throws -> [Track]
func getGenres() async throws -> [String]

// Search
func search(query: String, scope: SearchScope) async throws -> SearchResult
```

} ```

File System Service swift protocol FileSystemService { func getAvailableStorage() -> StorageInfo func getFileMetadata(at path: String) throws -> FileMetadata func moveFile(from: String, to: String) throws func deleteFile(at path: String) throws func createDirectory(at path: String) throws }

Metadata Service swift protocol MetadataService { func readMetadata(from file: URL) throws -> AudioMetadata func writeMetadata(to file: URL, metadata: AudioMetadata) throws func extractArtwork(from file: URL) throws -> UIImage? func embedArtwork(to file: URL, artwork: UIImage) throws func parseCueSheet(at path: String) throws -> [CueTrack] }

1. Database Schema Design

The database schema will use SwiftData with Core Data fallback, as outlined in the File System section. Key considerations:

- **Indexing Strategy:**
 - Create indexes on frequently queried fields: artist, album, title, genre
 - Implement full-text search indexes for search functionality
 - Use composite indexes for common query patterns

- **Migration Strategy:**

- Implement versioned schema migrations
- Support incremental updates without full library rescans
- Provide fallback mechanisms for schema changes

- **Library Import Flow**

1. User selects import source (Files app, external drive) 2. System scans selected location for supported audio files 3. For each file: a. Extract metadata using appropriate parser (ID3, Vorbis) b. Generate audio quality information (format, bit depth, sample rate) c. Extract or generate thumbnail for artwork d. Create database entries for track, album, artist 4. In background: a. Generate waveform data for visualization b. Process full-size artwork c. Calculate ReplayGain information (if enabled) 5. Update library views with new content 6. Notify user of import completion

1. Library Browsing Implementation

View Hierarchy - LibraryView : Main container with navigation to different views -

ArtistListView : Displays artists with artwork and track counts - AlbumListView : Displays albums with artwork, artist, and year - TrackListView : Displays tracks with metadata and quality indicators - GenreListView : Displays genres with associated track counts

ViewModels ```swift class LibraryViewModel: ObservableObject { @Published var artists: [Artist] = [] @Published var albums: [Album] = [] @Published var tracks: [Track] = [] @Published var genres: [String] = [] @Published var isLoading: Bool = false @Published var error: Error? = nil

```
private let libraryManager: LibraryManagerService
```

```
// Methods for loading and filtering library content
```

```
func loadArtists(sortBy: ArtistSortOption = .name) async
```

```
func loadAlbums(sortBy: AlbumSortOption = .title, filterBy: AlbumFilter? = nil) async
```

```
func loadTracks(sortBy: TrackSortOption = .title, filterBy: TrackFilter? = nil) async
```

```
func loadGenres() async
```

```
func search(query: String, scope: SearchScope = .all) async
```

```
} ```
```

1. Performance Optimization Techniques for Large Libraries

2. Indexing and Database Optimization:

- Create composite indexes for frequently queried combinations (e.g., artist+album, genre+year)
- Implement SQLite FTS5 for full-text search if SwiftData shows performance limitations
- Use batch operations with optimal batch sizes (500 items) for database operations
- Implement database sharding for libraries exceeding 50,000 tracks
- Create separate indexes for audio quality and format filtering

3. Lazy Loading and Pagination:

- Load only essential metadata during initial library scan
- Defer waveform generation and full artwork processing to background tasks
- Implement pagination with cursor-based approach for large result sets
- Use virtualized lists with cell recycling for smooth scrolling
- Implement progressive loading of complex metadata

4. Caching Strategy:

- Cache album artwork at multiple resolutions using NSCache with size limits
- Implement tiered caching system (memory → disk → regenerate)
- Cache frequently accessed metadata in memory with LRU eviction policy
- Persist processed data (waveforms, analyzed metadata) to avoid reprocessing
- Implement cache warming for frequently accessed items

5. Background Processing:

- Use `BackgroundTasks` framework for library maintenance
- Implement priority queue for background operations with task cancellation
- Provide user controls to pause/resume intensive operations
- Use `Task.detached` with appropriate priority levels for CPU-intensive operations
- Implement progressive processing with checkpointing for large operations

6. Benchmarking and Monitoring:

- Create synthetic test libraries (10k, 50k, 100k tracks) for performance testing
- Implement performance logging for critical operations
- Add diagnostics for query latency and file I/O operations
- Create performance profiles for different library sizes
- Monitor memory usage and implement automatic resource cleanup

7. Error Handling and Recovery

8. Implement robust error handling for file access issues
9. Provide recovery mechanisms for corrupted metadata
10. Log detailed error information for troubleshooting
11. Present user-friendly error messages with actionable steps
12. Implement automatic retry mechanisms for transient errors

13. Testing Strategy

14. Unit Tests:

- Test metadata parsing with various file formats
- Test database operations and migrations
- Test filtering and sorting logic

15. Integration Tests:

- Test end-to-end import process
- Test library browsing with large datasets
- Test search functionality with various queries

16. Performance Tests:

- Measure import speed with large libraries
- Test scrolling performance in library views
- Benchmark search operations

Feature 2: Advanced Playback Engine

Feature Goal

Provide a high-fidelity audio playback system capable of bit-perfect reproduction with extensive format support and audio enhancement capabilities, prioritizing sound quality while offering performance mode options.

API Relationships

- Interfaces with AVFoundation for primary playback
- Utilizes SFBAudioEngine for DSD and APE support
- Leverages FFmpegKit for additional codec support
- Integrates with iOS audio session for background playback
- Connects with hardware detection services for external DACs

Detailed Feature Requirements

1. High-Fidelity Playback

2. Support lossless, gapless playback for all supported formats
3. Prioritize bit-perfect output when supported by hardware
4. Implement high-resolution format support (up to 24-bit/192kHz)
5. Provide waveform visualization synchronized with playback
6. Display quality indicators and format badges during playback
7. Support for DSD playback (future phase)

8. Audio Processing

9. Implement 10-band equalizer with presets
10. Provide gain control and ReplayGain support
11. Support crossfade between tracks (configurable)
12. Implement sample rate conversion for DAC compatibility
13. Offer different performance modes (Balanced, Maximum Quality, Battery Saver)

14. Playback Controls

15. Support standard playback controls (play, pause, skip, seek)
16. Implement advanced controls (repeat, shuffle, queue management)
17. Provide scrubbing through waveform visualization
18. Support background playback with lock screen controls
19. Integrate with Control Center and headphone remote controls

20. Queue Management

21. Implement playback queue with drag-and-drop reordering
22. Support saving queue as playlist
23. Provide history of played tracks
24. Implement "play next" and "add to queue" functionality
25. Support clearing and shuffling queue

26. Hardware Integration

27. Detect and configure for external DACs
28. Adjust output settings based on connected hardware
29. Display hardware information (connected DAC, sample rate, bit depth)
30. Support bit-perfect mode when compatible hardware is detected
31. Handle audio route changes (headphones, speakers, Bluetooth)

Detailed Implementation Guide

1. System Architecture

Audio Engine Service ````swift protocol AudioEngineService { // Playback control func play(track: Track) async throws func playCollection(tracks: [Track], startIndex: Int) async throws func pause() func resume() func stop() func seekTo(position: TimeInterval)

```
// Queue management
func getQueue() -> [Track]
func addToQueue(track: Track)
func addToQueue(tracks: [Track])
func removeFromQueue(at index: Int)
func moveInQueue(from: Int, to: Int)
func clearQueue()

// Audio processing
func setEqualizerBands(_ bands: [Float])
func applyEqualizerPreset(_ preset: EqualizerPreset)
func setGain(_ gain: Float)
func enableReplayGain(_ enabled: Bool)
func setCrossfade(_ duration: TimeInterval)

// Performance mode
func setPerformanceMode(_ mode: PerformanceMode)

// Hardware configuration
func configureForConnectedHardware()
func getHardwareInfo() -> HardwareInfo
```

```
} ````
```

Audio Session Service swift protocol AudioSessionService { func configureAudioSession(category: AVAudioSession.Category, mode: AVAudioSession.Mode) func activateAudioSession() throws func deactivateAudioSession() throws func handleInterruption(_ notification: Notification) func handleRouteChange(_ notification: Notification) }

Waveform Service swift protocol WaveformService { func generateWaveform(for track: Track) async throws -> WaveformData func getWaveformData(for track: Track) async throws -> WaveformData? func clearWaveformCache() }

1. Audio Engine Implementation

The audio engine will use a multi-layered approach:

- **Primary Layer:** AVAudioEngine for standard formats and iOS integration

- **Extended Layer:** SFBAudioEngine for high-resolution and DSD formats
- **Fallback Layer:** FFmpegKit for additional codec support

Engine Selection Logic 1. Analyze track format, bit depth, and sample rate 2. If format is standard (MP3, AAC, ALAC) and within AVAAudioEngine capabilities: a. Use AVAAudioEngine for playback 3. Else if format is high-resolution or specialized (FLAC, DSD): a. If SFBAudioEngine supports the format: i. Use SFBAudioEngine for playback b. Else: i. Use FFmpegKit to decode to PCM ii. Pass PCM to AVAAudioEngine for playback 4. Configure output format based on: a. Connected hardware capabilities b. User's performance mode selection

1. Performance Modes Implementation

```
swift enum PerformanceMode { case balanced // Default: High-quality + battery-efficient case maximumQuality // No compression, full waveform, high-res case batterySaver // Lower resolution, simplified visuals }
```

Mode Configuration `` ` Balanced Mode: - Use native format decoding when possible - Generate medium-resolution waveforms - Apply moderate buffer sizes - Enable hardware acceleration when available

Maximum Quality Mode: - Always use bit-perfect playback when supported - Disable any lossy processing - Generate high-resolution waveforms - Use larger buffer sizes for stability - Prioritize audio fidelity over battery life

Battery Saver Mode: - Downsample high-resolution audio when playing - Generate simplified waveforms or disable visualization - Use smaller buffer sizes - Optimize for minimal CPU usage - Apply more aggressive caching `` `

1. Waveform Visualization Implementation

Waveform Generation `` ` 1. For each audio file: a. Decode audio data in chunks b. Calculate peak and RMS values for each chunk c. Apply delta encoding compression to reduce storage size d. Store values in binary compressed format (Base64 encoded) e. Implement downsampling for high-resolution files (24-bit/192kHz) f. Use simplified peak-only representation for DSD files g. Cache results for future use

1. Visualization rendering: a. In Library view: show simplified waveform or static preview b. In Playback view: render detailed waveform with current position c. Support zoom levels based on performance mode d. Color-code waveform based on audio quality e. Adapt resolution and detail based on performance mode: - Maximum Quality: Full resolution, real-time updates - Balanced: Medium

resolution, optimized updates - Battery Saver: Low resolution or static representation ````

Waveform View ```` swift struct WaveformView: View { let waveformData: WaveformData let currentPosition: TimeInterval let duration: TimeInterval let isPlaying: Bool let quality: AudioQuality

```
// Interaction handlers
var onSeek: (TimeInterval) -> Void

// View implementation with Metal or SwiftUI Canvas
// for efficient rendering
```

} ````

1. Hardware Integration

DAC Detection and Configuration ```` 1. Monitor audio route changes via AVAudioSession 2. When external device is connected: a. Identify device type and capabilities b. Determine supported formats, sample rates, and bit depths c. Configure audio engine output format accordingly d. Update UI to show connected device information e. Enable bit-perfect mode if supported f. Validate bit-perfect output using BitPerfectValidatorService

1. When device is disconnected: a. Reconfigure for internal speaker or headphone output b. Adjust quality settings as needed ````

BitPerfectValidatorService ```` swift protocol BitPerfectValidatorService { // Validate if current output is truly bit-perfect func validateBitPerfectOutput(for track: Track) async -> BitPerfectValidationResult

```
// Get hardware capability information
func getHardwareCapabilities() -> HardwareCapabilities

// Check if specific format can be output bit-perfectly
func canOutputBitPerfect(format: AudioFormat) -> Bool

// Maintain compatibility database
func getDACCompatibilityInfo(for deviceID: String) -> DACCompatibilityInfo?

// Update compatibility database based on validation results
func updateDACCompatibilityInfo(for deviceID: String, result: BitPerfectValidationResult)
```

}

```
struct BitPerfectValidationResult { let isValid: Bool let actualSampleRate: Double let
actualBitDepth: Int let mismatchReason: String? let recommendedSettings:
AudioSettings? }
```

```
struct DACCompatibilityInfo { let deviceName: String let supportedSampleRates:
[Double] let supportedBitDepths: [Int] let requiresExclusiveMode: Bool let knownIssues:
[String]? let recommendedSettings: AudioSettings? } ````
```

Hardware Info Display ```` swift struct HardwareInfoView: View { let connectedDevice:
AudioDevice? let currentFormat: AudioFormat let isBitPerfect: Bool

```
// View implementation showing device name, format,
// sample rate, bit depth, and bit-perfect indicator
```

```
} ````
```

1. Audio Processing Pipeline

DSP Chain ```` Input → Format Decoder → [Optional Processing] → Output

Optional Processing modules (implemented in phases):

Phase 1 (MVP): - Equalizer (10-band) using AVAAudioUnitEQ - Gain Control (basic volume adjustment) - Sample Rate Conversion (for hardware compatibility)

Phase 2: - ReplayGain support - Crossfade between tracks - Enhanced equalizer with custom presets

Phase 3: - Advanced DSP effects - Spatial audio processing - Room correction

Each module can be enabled/disabled based on: - User preferences - Performance mode - Hardware capabilities ````

Equalizer Implementation ```` swift class EqualizerService { // 10 bands: 32Hz, 64Hz, 125Hz, 250Hz, 500Hz, 1kHz, 2kHz, 4kHz, 8kHz, 16kHz private var bands: [Float] = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0] private var isEnabled: Bool = false

```
// Methods to configure equalizer
func setBands(_ values: [Float])
func applyPreset(_ preset: EqualizerPreset)
func enable(_ enabled: Bool)
```

```
// Internal methods to apply EQ to audio stream
private func applyEqualization(buffer: AVAAudioPCMBuffer)
```

```
} `` `
```

1. Background Playback

Audio Session Configuration `` ` swift func configureForBackgroundPlayback() { do
{ try audioSessionService.configureAudioSession(category: .playback, mode: .default)
try audioSessionService.activateAudioSession()

```
    // Register for notifications
    NotificationCenter.default.addObserver(
        self,
        selector: #selector(handleInterruption),
        name: AVAudioSession.interruptionNotification,
        object: nil
    )

    NotificationCenter.default.addObserver(
        self,
        selector: #selector(handleRouteChange),
        name: AVAudioSession.routeChangeNotification,
        object: nil
    )

    // Configure Now Playing info
    setupNowPlayingInfo()

    // Configure remote command center
    setupRemoteCommandCenter()
} catch {
    logger.error("Failed to configure audio session: \(error)")
}
```

```
} `` `
```

1. Error Handling and Recovery

2. Implement robust error handling for audio decoding issues
3. Provide fallback mechanisms for unsupported formats
4. Handle audio session interruptions gracefully
5. Implement automatic recovery from playback errors
6. Log detailed error information for troubleshooting

7. Testing Strategy

8. Unit Tests:

- Test format detection and decoder selection
- Test equalizer and audio processing modules
- Test queue management logic

9. Integration Tests:

- Test end-to-end playback with various formats
- Test hardware detection and configuration
- Test background playback and controls

10. Performance Tests:

- Measure CPU and memory usage during playback
- Test battery consumption in different modes
- Benchmark waveform rendering performance

Feature 3: Smart Playlist & Filtering

Feature Goal

Provide a powerful system for creating dynamic playlists based on metadata attributes, allowing users to organize their music collection with precision and flexibility.

API Relationships

- Interfaces with Library Manager for track access
- Utilizes SwiftData query predicates for filtering
- Connects with Playback Engine for playlist playback
- Leverages Combine framework for reactive updates

Detailed Feature Requirements

1. Smart Playlist Creation

2. Support creating playlists with multiple filter criteria
3. Implement complex boolean logic (AND, OR, NOT)
4. Allow filtering by audio quality, genre, artist, and custom tags
5. Support numerical comparisons (greater than, less than, equal to)
6. Allow date-based filtering (added, modified, last played)
7. Support text matching (contains, starts with, exact match)

8. Filter Management

9. Save and manage multiple playlist configurations
10. Support editing existing smart playlists
11. Allow duplicating and modifying playlists
12. Provide preset filters for common scenarios
13. Support importing and exporting filter configurations

14. **Result Presentation**

15. Sort and group results by various attributes
16. Provide visual indicators for filter matches
17. Support real-time updates as library changes
18. Allow switching between different view modes
19. Implement pagination for large result sets

20. **Quick Filtering**

21. Support filtering within current view context
22. Provide quick filter presets (high-res only, recently added, etc.)
23. Implement search within filtered results
24. Allow temporary modifications to filters
25. Support saving quick filters as smart playlists

26. **Performance Optimization**

27. Optimize query execution for large libraries
28. Implement caching for frequent queries
29. Use background processing for complex filters
30. Support incremental updates to filtered results
31. Provide progress indicators for long-running filters

Detailed Implementation Guide

1. System Architecture

Smart Playlist Service ```swift protocol SmartPlaylistService { // Playlist management
func createSmartPlaylist(name: String, rules: [FilterRule]) async throws -> Playlist
func updateSmartPlaylist(id: UUID, name: String?, rules: [FilterRule]?) async throws -> Playlist
func deleteSmartPlaylist(id: UUID) async throws
func getSmartPlaylists() async throws -> [Playlist]
func getSmartPlaylist(id: UUID) async throws -> Playlist?

```
// Filter execution  
func executeFilter(rules: [FilterRule]) async throws -> [Track]
```

```
func executeQuickFilter(rule: FilterRule, in tracks: [Track]) -> [Track]
```

```
// Preset management
```

```
func saveFilterPreset(name: String, rules: [FilterRule]) async throws
```

```
func getFilterPresets() async throws -> [FilterPreset]
```

```
} `` `
```

Filter Rule Model `` ` swift struct FilterRule { enum Attribute { case title, artist, albumTitle, albumArtist, genre, composer case year, trackNumber, discNumber, duration, playCount case format, bitDepth, sampleRate, bitrate case isLossless, isHiRes case dateAdded, dateModified, lastPlayed case custom(String) }

```
enum Operator {  
    // Text operators  
    case contains, doesNotContain, beginsWith, endsWith, equals, notEquals
```

```
    // Numeric operators  
    case greaterThan, lessThan, equalTo, notEqualTo  
    case between, notBetween
```

```
    // Boolean operators  
    case isTrue, isFalse
```

```
    // Date operators  
    case before, after, on  
    case inTheLast, notInTheLast
```

```
}
```

```
enum Combinator {  
    case and, or  
}
```

```
var attribute: Attribute
```

```
var `operator`: Operator
```

```
var value: Any
```

```
var secondaryValue: Any? // For range operators like "between"
```

```
var combinator: Combinator? // How this rule combines with the next rule
```

```
} `` `
```

1. Database Schema Design

The smart playlist functionality will leverage the existing database schema with additional models for storing filter rules:

```
```swift @Model class PlaylistEntity { var id: UUID var name: String var dateCreated: Date var dateModified: Date var isSmartPlaylist: Bool var smartRules: Data? // JSON serialized rules for smart playlists
```

```
 // Relationships
 var tracks: [PlaylistTrackEntity]?
```

```
}
```

```
@Model class FilterPresetEntity { var id: UUID var name: String var rules: Data // JSON serialized filter rules var dateCreated: Date var dateModified: Date } ```
```

**Rule Serialization** ```swift extension FilterRule { func toJSON() -> Data? { // Convert rule to JSON representation }

```
 static func fromJSON(_ data: Data) -> FilterRule? {
 // Create rule from JSON representation
 }
```

```
} ```
```

## 1. Filter Execution Engine

**Query Builder** ```swift class QueryBuilder { // Build SwiftData predicates from filter rules func buildPredicate(from rules: [FilterRule]) -> NSPredicate { var predicates: [NSPredicate] = [] var currentCombinator: FilterRule.Combinator = .and

```
 for rule in rules {
 let predicate = buildSinglePredicate(from: rule)
 predicates.append(predicate)

 if let nextCombinator = rule.combinator {
 currentCombinator = nextCombinator
 }
 }

 if currentCombinator == .and {
 return NSCompoundPredicate(andPredicateWithSubpredicates: predicates)
 } else {
 return NSCompoundPredicate(orPredicateWithSubpredicates: predicates)
 }
}

private func buildSinglePredicate(from rule: FilterRule) -> NSPredicate {
 // Build predicate based on attribute, operator, and value
 switch rule.attribute {
```

```

 case .title:
 return buildStringPredicate(keyPath: "title", operator: rule.operator, value:
rule.value)
 case .artist:
 return buildStringPredicate(keyPath: "artist", operator: rule.operator, value:
rule.value)
 // Additional cases for other attributes
 }
}

private func buildStringPredicate(keyPath: String, operator: FilterRule.Operator,
value: Any) -> NSPredicate {
 // Build string-specific predicates
 switch `operator` {
 case .contains:
 return NSPredicate(format: "%K CONTAINS[cd] %@", keyPath, value as!
String)
 case .doesNotContain:
 return NSPredicate(format: "NOT (%K CONTAINS[cd] %@", keyPath, value
as! String)
 // Additional cases for other operators
 }
}

// Additional methods for building numeric, boolean, and date predicates

```

```

} `` `

```

## 1. Smart Playlist Editor UI

**View Hierarchy** - SmartPlaylistEditorView : Main editor view - FilterRuleView : Individual rule editor - AttributeSelectorView : Attribute selection - OperatorSelectorView : Operator selection - ValueEditorView : Value input based on attribute type

**ViewModel** `` ` swift class SmartPlaylistEditorViewModel: ObservableObject  
{ @Published var playlistName: String = "" @Published var rules: [FilterRule] = []  
@Published var matchingTracks: [Track] = [] @Published var isExecutingFilter: Bool =  
false @Published var error: Error? = nil

```

private let smartPlaylistService: SmartPlaylistService
private var cancellables = Set<AnyCancellable>()

// Methods for managing rules
func addRule()
func removeRule(at index: Int)
func updateRule(at index: Int, rule: FilterRule)
func moveRule(from: Int, to: Int)

// Methods for filter execution

```



```

func executeFilter() async
func savePlaylist() async throws -> Playlist
func updatePlaylist(id: UUID) async throws -> Playlist

// Helper methods
func getAvailableAttributes() -> [FilterRule.Attribute]
func getOperators(for attribute: FilterRule.Attribute) -> [FilterRule.Operator]

```

```

} `` `

```

## 1. Real-Time Filtering Implementation

**Reactive Updates** `` ` swift **func** setupReactiveUpdates() { // Observe changes to rules and execute filter \$rules .debounce(for: .seconds(0.5), scheduler: RunLoop.main) .sink { [weak self] \_ in Task { await self?.executeFilter() } } .store(in: &cancellables)

```

// Observe library changes and update results if needed
NotificationCenter.default
 .publisher(for: .libraryDidUpdate)
 .debounce(for: .seconds(1.0), scheduler: RunLoop.main)
 .sink { [weak self] _ in
 Task {
 await self?.executeFilter()
 }
 }
 .store(in: &cancellables)

```

```

} `` `

```

## 1. Quick Filtering Implementation

**Quick Filter Service** `` ` swift **class** QuickFilterService { // Predefined quick filters static let highResOnly = FilterRule( attribute: .isHiRes, operator: .isTrue, value: true )

```

static let recentlyAdded = FilterRule(
 attribute: .dateAdded,
 operator: .inTheLast,
 value: 7,
 secondaryValue: "days"
)

static let neverPlayed = FilterRule(
 attribute: .playCount,
 operator: .equalTo,
 value: 0
)

// Apply quick filter to current view

```

```
func applyQuickFilter(_ rule: FilterRule, to tracks: [Track]) -> [Track]

// Save quick filter as smart playlist
func saveAsSmartPlaylist(name: String, rule: FilterRule) async throws -> Playlist
```

```
} `` `
```

## 1. Performance Optimization Techniques

### 2. Query Optimization:

- Use indexed fields for filtering when possible
- Optimize predicate structure for efficient execution
- Limit result set size for initial preview

### 3. Caching Strategy:

- Cache filter results for frequently used smart playlists
- Implement incremental updates instead of full re-execution
- Store intermediate results for complex filters

### 4. Background Processing:

- Execute complex filters in background tasks
- Update UI with progress indicators for long-running filters
- Allow cancellation of filter execution

## 5. Error Handling and Recovery

6. Implement validation for filter rules to prevent invalid combinations

7. Provide clear error messages for filter execution failures

8. Handle edge cases like empty results or too many results

9. Implement automatic simplification for overly complex filters

10. Log detailed error information for troubleshooting

## 11. Testing Strategy

### 12. Unit Tests:

- Test predicate building for various rule combinations
- Test filter execution with different data sets
- Test rule serialization and deserialization

### 13. Integration Tests:

- Test end-to-end smart playlist creation and execution
- Test performance with large libraries
- Test reactive updates with library changes

### 14. User Testing:

- Validate filter UI usability
- Test complex filter creation workflows
- Measure time to create common filter patterns

## Feature 4: Metadata Editor

### Feature Goal

Provide a comprehensive tool for viewing and editing audio file metadata, including tags, artwork, and lyrics, with batch editing capabilities to efficiently manage large music collections.

### API Relationships

- Interfaces with TagLib for metadata reading/writing
- Connects with Library Manager for track access
- Utilizes File System Service for file operations
- Integrates with Background Task Service for batch operations

### Detailed Feature Requirements

#### 1. Metadata Viewing and Editing

2. Support viewing and editing ID3, Vorbis, and other tag formats
3. Provide fields for common metadata (title, artist, album, etc.)
4. Support extended metadata fields (composer, conductor, etc.)
5. Allow editing custom tag fields

6. Implement validation for metadata fields

#### 7. Artwork Management

8. Display embedded album artwork
9. Support adding, replacing, and removing artwork
10. Handle high-resolution artwork appropriately
11. Support multiple artwork (front cover, back cover, etc.)
12. Provide image editing capabilities (crop, resize)

### 13. Lyrics Support

- 14. View and edit embedded lyrics
- 15. Support synchronized lyrics (LRC format)
- 16. Allow importing lyrics from text files
- 17. Provide lyrics display during playback
- 18. Support multiple languages for lyrics

### 19. Batch Editing

- 20. Edit metadata for multiple files simultaneously
- 21. Support pattern-based field editing
- 22. Implement find and replace functionality
- 23. Allow applying artwork to multiple albums
- 24. Provide preview of changes before applying

### 25. Advanced Features

- 26. Support for cue sheets and track splitting
- 27. Implement metadata extraction from filenames
- 28. Provide filename formatting based on metadata
- 29. Support for album-level and track-level metadata
- 30. Implement undo/redo functionality for edits

## Detailed Implementation Guide

### 1. System Architecture

**Metadata Editor Service** ```swift protocol MetadataEditorService { // Single file operations func getMetadata(for track: Track) async throws -> AudioMetadata func saveMetadata(for track: Track, metadata: AudioMetadata) async throws

```
// Artwork operations
func getArtwork(for track: Track) async throws -> [Artwork]
func saveArtwork(for track: Track, artwork: Artwork) async throws
func removeArtwork(for track: Track) async throws
```

```
// Lyrics operations
func getLyrics(for track: Track) async throws -> Lyrics?
func saveLyrics(for track: Track, lyrics: Lyrics) async throws
func removeLyrics(for track: Track) async throws
```

```
// Batch operations
func batchEdit(tracks: [Track], changes: MetadataChanges) async throws ->
BatchEditResult
```

```
func previewBatchEdit(tracks: [Track], changes: MetadataChanges) async throws -
> BatchEditPreview

// Advanced operations
func extractMetadataFromFilename(track: Track, pattern: String) async throws ->
AudioMetadata
func formatFilename(track: Track, pattern: String) async throws -> String
func splitCueSheet(cueFile: URL) async throws -> [Track]
```

```
} `` `
```

**Audio Metadata Model** `` ` swift struct AudioMetadata { // Common metadata var title: String? var artist: String? var albumTitle: String? var albumArtist: String? var composer: String? var conductor: String? var genre: String? var year: Int? var trackNumber: Int? var trackTotal: Int? var discNumber: Int? var discTotal: Int? var comment: String?

```
// Extended metadata
var bpm: Int?
var key: String?
var mood: String?
var recordLabel: String?
var catalogNumber: String?
var isrc: String?
var language: String?
var copyright: String?
var encodedBy: String?
var encodingSettings: String?

// Custom fields
var customFields: [String: String]
```

```
} `` `
```

**Artwork Model** `` ` swift struct Artwork { enum ArtworkType { case frontCover case backCover case disc case artist case liner case other }

```
var type: ArtworkType
var image: UIImage
var description: String?
var mimeType: String
```

```
} `` `
```

**Lyrics Model** `` ` swift struct Lyrics { enum LyricsType { case unsynchronized case synchronized }

```

var type: LyricsType
var content: String
var language: String?

// For synchronized lyrics
var timeMarkers: [TimeMarker]?

struct TimeMarker {
 var time: TimeInterval
 var text: String
}

```

```

} ``

```

## 1. Metadata Editor UI

**View Hierarchy** - MetadataEditorView : Main editor container - BasicMetadataView : Common metadata fields - ExtendedMetadataView : Additional metadata fields - ArtworkEditorView : Artwork management - LyricsEditorView : Lyrics editing - BatchEditorView : Batch editing interface

**ViewModel** `` swift class MetadataEditorViewModel: ObservableObject { @Published var metadata: AudioMetadata @Published var artwork: [Artwork] = [] @Published var lyrics: Lyrics? @Published var isEditing: Bool = false @Published var isSaving: Bool = false @Published var error: Error? = nil

```

private let track: Track
private let metadataService: MetadataEditorService

// Methods for metadata operations
func loadMetadata() async
func saveMetadata() async throws
func revertChanges()

// Methods for artwork operations
func loadArtwork() async
func addArtwork(_ image: UIImage, type: Artwork.ArtworkType)
func removeArtwork(at index: Int)
func saveArtwork() async throws

// Methods for lyrics operations
func loadLyrics() async
func saveLyrics() async throws
func importLyrics(from url: URL) async throws
func convertToSynchronized() -> Lyrics

```

```
// Validation methods
func validateMetadata() -> [ValidationError]
```

```
} `` `
```

## 1. Batch Editing Implementation

**Batch Changes Model** `` ` swift struct MetadataChanges { enum ChangeType { case set(value: Any) case clear case append(value: String) case prepend(value: String) case replace(search: String, replacement: String) case increment(start: Int, step: Int) case capitalize case lowercase case uppercase }

```
var changes: [KeyPath<AudioMetadata, Any>: ChangeType]
var artworkChange: ArtworkChange?
var lyricsChange: LyricsChange?

enum ArtworkChange {
 case set(artwork: Artwork)
 case clear
 case keepExisting
}

enum LyricsChange {
 case set(lyrics: Lyrics)
 case clear
 case keepExisting
}
```

```
} `` `
```

**Batch Editor ViewModel** `` ` swift class BatchEditorViewModel: ObservableObject { @Published var selectedTracks: [Track] = [] @Published var changes: MetadataChanges = MetadataChanges(changes: [:]) @Published var previewResults: BatchEditPreview? @Published var isExecuting: Bool = false @Published var progress: Float = 0.0 @Published var error: Error? = nil

```
private let metadataService: MetadataEditorService

// Methods for batch operations
func previewChanges() async
func applyChanges() async throws
func cancelChanges()

// Helper methods
func addChange(for keyPath: KeyPath<AudioMetadata, Any>, type: MetadataChanges.ChangeType)
func removeChange(for keyPath: KeyPath<AudioMetadata, Any>)
```

```
func setArtworkChange(_ change: MetadataChanges.ArtworkChange)
func setLyricsChange(_ change: MetadataChanges.LyricsChange)
```

```
} `` `
```

## 1. Tag Writing Implementation

**TagLib Integration** `` ` swift class TagLibService { // Read metadata from file func readTags(from url: URL) throws -> AudioMetadata { // Use TagLib to read metadata from file // Map TagLib structures to AudioMetadata model }

```
 // Write metadata to file
 func writeTags(to url: URL, metadata: AudioMetadata) throws {
 // Use TagLib to write metadata to file
 // Handle different tag formats based on file type
 }
```

```
 // Read artwork from file
 func readArtwork(from url: URL) throws -> [Artwork] {
 // Extract artwork from file using TagLib
 // Convert to Artwork model
 }
```

```
 // Write artwork to file
 func writeArtwork(to url: URL, artwork: [Artwork]) throws {
 // Embed artwork in file using TagLib
 // Handle different formats and sizes
 }
```

```
 // Read lyrics from file
 func readLyrics(from url: URL) throws -> Lyrics? {
 // Extract lyrics from file using TagLib
 // Parse synchronized lyrics if available
 }
```

```
 // Write lyrics to file
 func writeLyrics(to url: URL, lyrics: Lyrics) throws {
 // Embed lyrics in file using TagLib
 // Format synchronized lyrics if needed
 }
```

```
} `` `
```

## 1. Non-Destructive Editing

```
`` ` swift class BackupService { // Create backup of file before editing func backupFile(at url: URL) throws -> URL { // Create backup copy with timestamp // Store in backup directory // Return backup URL }
```



```

// Restore file from backup
func restoreFile(from backupURL: URL, to originalURL: URL) throws {
 // Copy backup file back to original location
 // Verify integrity
}

// Clean up old backups
func cleanupBackups(olderThan date: Date) throws {
 // Remove backups older than specified date
 // Keep minimum number of backups
}

```

```

} `` `

```

## 1. CUE Sheet Handling

```

`` `swift class CueSheetService { // Parse CUE sheet
func parseCueSheet(at url: URL)
throws -> CueSheet { // Parse CUE file format // Extract track information // Map to
CueSheet model }

```

```

// Split audio file based on CUE sheet
func splitAudioFile(audioURL: URL, cueSheet: CueSheet, outputDirectory: URL)
async throws -> [URL] {
 // Use FFmpeg to split audio file
 // Apply metadata from CUE sheet
 // Return URLs of created files
}

// Create CUE sheet from tracks
func createCueSheet(for tracks: [Track], outputURL: URL) throws {
 // Generate CUE sheet format
 // Include track metadata
 // Write to file
}

```

```

} `` `

```

## 1. Performance Optimization Techniques

### 2. Batch Processing:

- Process files in batches to limit memory usage
- Implement progress tracking for long operations
- Allow cancellation of batch operations

### **3. Background Execution:**

- Use background tasks for intensive operations
- Implement resumable operations for large batches
- Provide notifications for completed operations

### **4. Caching Strategy:**

- Cache metadata and artwork during editing sessions
- Implement dirty tracking to only save changed fields
- Use memory-efficient representations for large artwork

### **5. Error Handling and Recovery**

6. Implement robust error handling for file access issues

7. Provide automatic backup before making changes

8. Implement rollback mechanisms for failed batch operations

9. Present user-friendly error messages with recovery options

10. Log detailed error information for troubleshooting

### **11. Testing Strategy**

#### **12. Unit Tests:**

- Test metadata reading and writing with various formats
- Test batch operation logic
- Test validation rules

#### **13. Integration Tests:**

- Test end-to-end metadata editing workflow
- Test batch editing with large collections
- Test CUE sheet parsing and splitting

#### **14. File Format Tests:**

- Test compatibility with various tag formats
- Test handling of corrupt or incomplete tags
- Test preservation of existing tags not modified by the editor

# Feature Integration Points

## Integration Between Core Music Library and Advanced Playback Engine

### 1. Track Selection and Playback

2. Library provides track metadata and file paths to Playback Engine
3. Playback Engine reports playback status and position back to Library
4. Library updates play counts and last played timestamps

### 5. Audio Quality Information

6. Library provides format, bit depth, and sample rate information
7. Playback Engine configures appropriate decoder based on format
8. Library displays quality indicators based on playback capabilities

### 9. Waveform Visualization

10. Library initiates background waveform generation for new tracks
11. Playback Engine uses waveform data for visualization during playback
12. Library caches waveform data for future use

## Integration Between Core Music Library and Smart Playlist & Filtering

### 1. Library Access

2. Smart Playlist feature queries Library for tracks matching filter criteria
3. Library notifies Smart Playlist when tracks are added, removed, or modified
4. Smart Playlist updates filtered results based on library changes

### 5. Metadata Filtering

6. Library provides metadata attributes for filtering
7. Smart Playlist creates predicates based on these attributes
8. Library executes queries and returns matching tracks

### 9. Performance Optimization

10. Library provides indexing for efficient filtering
11. Smart Playlist optimizes queries for performance
12. Both components coordinate background processing

## Integration Between Core Music Library and Metadata Editor

### 1. Track Selection

2. Library provides tracks for editing in Metadata Editor
3. Metadata Editor reads and writes metadata for selected tracks
4. Library updates its database after metadata changes

### 5. Batch Operations

6. Library provides collections of tracks for batch editing
7. Metadata Editor applies changes to multiple files
8. Library updates all affected entries after batch operations

### 9. File System Coordination

10. Library manages file paths and organization
11. Metadata Editor works with files at these paths
12. Both components coordinate file access to prevent conflicts

## Integration Between Advanced Playback Engine and Smart Playlist & Filtering

### 1. Playlist Playback

2. Smart Playlist provides filtered track lists to Playback Engine
3. Playback Engine manages playback queue based on these lists
4. Smart Playlist updates can modify current playback queue

### 5. Playback Statistics

6. Playback Engine reports play counts and durations
7. Smart Playlist can filter based on these statistics
8. Both components share playback history data

## Integration Between Advanced Playback Engine and Metadata Editor

### 1. Real-time Metadata Display

2. Playback Engine displays current track metadata during playback
3. Metadata Editor can update this information in real-time
4. Playback Engine reflects metadata changes immediately

### 5. Lyrics Display

6. Metadata Editor manages lyrics content and timing
7. Playback Engine displays synchronized lyrics during playback
8. Both components share lyrics parsing and formatting logic

## **Integration Between Smart Playlist & Filtering and Metadata Editor**

### **1. Metadata-based Filtering**

2. Metadata Editor updates track attributes
3. Smart Playlist reevaluates filters based on updated attributes
4. Both components share metadata validation logic

### **5. Batch Operations**

6. Smart Playlist can select tracks for batch editing
7. Metadata Editor applies changes to these selections
8. Smart Playlist updates after batch operations complete

## **Performance Considerations**

### **1. Large Library Optimization**

2. Implement pagination for large result sets
3. Use background indexing for library scanning
4. Optimize database queries with appropriate indexes
5. Implement lazy loading for artwork and detailed metadata

### **6. Audio Playback Efficiency**

7. Use appropriate buffer sizes based on performance mode
8. Implement efficient format conversion when necessary
9. Optimize waveform rendering for smooth visualization
10. Minimize processing during battery-saving mode

### **11. Background Processing**

12. Coordinate background tasks to prevent CPU overload
13. Prioritize playback performance over background operations
14. Implement cancellation for non-critical background tasks
15. Use low-priority queues for metadata processing

### **16. Memory Management**

17. Implement proper caching strategies for artwork and waveforms
18. Release unused resources when memory pressure occurs
19. Use appropriate image resolutions based on display requirements
20. Implement memory-efficient data structures for large collections

## 21. **Battery Optimization**

22. Provide user-selectable performance modes
23. Reduce processing during battery-saving mode
24. Optimize background operations when on battery power
25. Implement efficient sleep/wake behavior

# Security and Privacy Considerations

## 1. **Privacy-First Design Philosophy**

2. Implement strict privacy-by-default approach with no data collection
3. Create privacy onboarding screen explaining the app's privacy commitment
4. Provide detailed privacy policy accessible from settings
5. Implement privacy indicators for any feature that could potentially access external services
6. Use anonymous identifiers if any analytics are implemented (opt-in only)

## 7. **File System Access**

8. Use proper sandboxing for file access with explicit user permissions
9. Implement secure file operations with comprehensive error handling
10. Provide clear user permissions dialogs with explanations
11. Protect backup files with appropriate permissions and optional encryption
12. Implement secure deletion for sensitive files when requested

## 13. **Metadata Privacy**

14. Keep all metadata processing strictly on-device with no external transmission
15. Implement secure local storage for all metadata with optional encryption
16. Never transmit library information without explicit user opt-in consent
17. Provide granular controls for what metadata can be shared if user enables external services
18. Implement metadata anonymization for any shared data (if user opts in)
19. Allow users to audit what metadata is stored and how it's used

## 20. External Service Integration

- 21. Implement OAuth 2.0 with PKCE for all external service authentication
- 22. Use encrypted connections (TLS 1.3+) for all network traffic
- 23. Minimize data sharing with external services to only what's necessary
- 24. Provide clear privacy controls with granular permissions for each service
- 25. Implement connection security indicators in the UI
- 26. Allow users to revoke access to external services at any time
- 27. Store authentication tokens securely in the iOS Keychain

## 28. User Data Protection

- 29. Implement secure storage for user preferences using iOS Keychain
- 30. Protect playback history and statistics with local-only storage
- 31. Provide options to disable history tracking entirely
- 32. Allow users to clear usage data with one-tap option
- 33. Comply with GDPR, CCPA, and other privacy regulations
- 34. Implement data portability for user-generated content
- 35. Use SecureEnclave for any sensitive cryptographic operations

## 36. Network Security

- 37. Implement certificate pinning for API connections
- 38. Use App Transport Security (ATS) with no exceptions
- 39. Implement network activity indicators for user awareness
- 40. Provide offline mode with clear indicators when network features are unavailable
- 41. Implement bandwidth usage monitoring and controls

# Accessibility Considerations

## 1. VoiceOver Support

- 2. Implement proper accessibility labels for all UI elements
- 3. Provide meaningful descriptions for audio quality indicators
- 4. Ensure logical navigation flow for screen readers
- 5. Test all features with VoiceOver enabled

## 6. Dynamic Type

- 7. Support dynamic text sizing throughout the app
- 8. Ensure layouts adapt to larger text sizes
- 9. Maintain readability at all text sizes

10. Test with various text size settings

### 11. **Reduce Motion**

12. Provide alternative animations for users with motion sensitivity

13. Implement static alternatives to waveform animations

14. Respect system settings for reduced motion

15. Test with reduced motion enabled

### 16. **Color and Contrast**

17. Ensure sufficient contrast for all UI elements

18. Do not rely solely on color for conveying information

19. Provide alternative indicators for color-blind users

20. Test with various color filters enabled

### 21. **Haptic Feedback**

22. Implement appropriate haptic feedback for interactions

23. Provide alternative feedback mechanisms

24. Respect system settings for haptic feedback

25. Test with various haptic settings